PROYECTO INTEGRADOR DE DATA SCIENCE

FleetLogix

Pipeline ETL y Data Warehouse para Gestión de Flotas

Documentación técnica del proceso ETL (PostgreSQL → Python → Snowflake),
el modelo dimensional en estrella y ejemplos de los datos cargados.

Julio 2026

RESUMEN DEL PROYECTO

1. ¿Qué es FleetLogix?

FleetLogix es el proyecto integrador de Data Science que consistió en construir un pipeline de datos completo para una empresa de logística y reparto de flotas de vehículos. El objetivo fue tomar la información operativa que se genera día a día en un sistema transaccional (PostgreSQL) — entregas, viajes, vehículos, conductores y rutas — y transformarla en un modelo analítico limpio y confiable dentro de un Data Warehouse en Snowflake, listo para que áreas como Operaciones y Ventas puedan construir reportes y tomar decisiones.

El trabajo se dividió en tres grandes bloques: el diseño del **modelo dimensional** (esquema en estrella), la construcción del **pipeline ETL** en Python que mueve y transforma los datos, y la creación de **vistas seguras por rol** para controlar qué puede ver cada área de la empresa.

3	7	2
Etapas del ETL (Extract–Transform–Load)	Tablas del modelo (6 dimensiones + 1 hechos)	Vistas seguras por rol de negocio

2. Arquitectura general

La arquitectura sigue un flujo clásico de ETL con tres capas bien diferenciadas:

Capa	Tecnología	Función
1. Origen	PostgreSQL (fleetlogix)	Base transaccional con las tablas operativas: deliveries, trips, vehicles, drivers y routes.
2. Procesamiento	Python (pandas, psycopg2, snowflake-connector)	Script A3-05_etl_pipeline: extrae los datos del día anterior, los limpia, calcula métricas y arma las claves necesarias para el modelo dimensional.
3. Destino	Snowflake (FLEETLOGIX_DW)	Data Warehouse en esquema estrella: 6 tablas de dimensión + 1 tabla de hechos, con vistas seguras para Ventas y Operaciones.

El pipeline está pensado para correr de forma automática todos los días a las 2:00 AM, procesando únicamente las entregas del día anterior con estado "delivered".

DISEÑO DE DATOS

3. Modelo dimensional (esquema en estrella)

En el centro del modelo está **fact_deliveries**, la tabla de hechos que guarda una fila por cada entrega realizada, con sus métricas (tiempos, costos, ingresos, kilómetros, litros de combustible) y las claves que la conectan con seis dimensiones descriptivas. Este diseño permite analizar las entregas desde múltiples ángulos — por fecha, por conductor, por vehículo, por ruta o por cliente — sin duplicar información.

Dimensión	Describe...	Atributos principales
dim_date	Fecha de la entrega	día de semana, mes, trimestre, año, feriados
dim_time	Hora de la entrega	hora, franja del día (madrugada/mañana/tarde/noche), turno
dim_vehicle	Vehículo utilizado	patente, tipo, capacidad, combustible, antigüedad, estado
dim_driver	Conductor asignado	nombre, licencia, antigüedad, categoría de desempeño
dim_route	Ruta recorrida	ciudad origen/destino, distancia, peajes, dificultad
dim_customer	Cliente que recibe	nombre, tipo (individual/empresa/gobierno), ciudad, categoría

La tabla de hechos **fact_deliveries** combina las siete claves (una por dimensión, más dos referencias de tiempo: hora programada y hora real de entrega) junto con las métricas operativas: peso del paquete, distancia, litros de combustible consumidos, minutos de entrega, minutos de atraso, entregas por hora, eficiencia de combustible, costo e ingreso por entrega, y los indicadores *is_on_time*, *is_damaged* y *has_signature*.

3.1 Vistas seguras por rol

Para que cada equipo acceda solo a lo que necesita, se crearon dos vistas seguras (*SECURE VIEW*) sobre la tabla de hechos:

Vista	Rol asignado	Contenido
v_sales_deliveries	SALES_ANALYST	Cliente, tipo de cliente, peso, estado e ingreso por entrega. Excluye clientes de tipo "Gobierno" por restricción de negocio.
v_operations_deliveries	OPERATIONS_ANALYST	Vista completa: fecha, hora, patente, conductor, ruta, cliente, tiempos de entrega, atrasos, puntualidad y combustible consumido.

PROCESO ETL

4. El pipeline ETL paso a paso

4.1 Extract — Extracción desde PostgreSQL

El script se conecta a la base *fleetlogix* en PostgreSQL y ejecuta una consulta que cruza las tablas *deliveries*, *trips*, *vehicles*, *drivers* y *routes*, trayendo únicamente las entregas del día anterior con estado "delivered". Esto asegura que el ETL procese siempre un lote acotado y consistente de datos ya finalizados.

4.2 Transform — Limpieza y cálculo de métricas

Esta es la etapa más importante del pipeline. Sobre los datos extraídos se aplican, entre otras, las siguientes transformaciones:

- Se convierten los campos de fecha/hora a formato datetime y se descartan filas sin fecha de entrega real.
- Se calcula *delivery_time_minutes* (diferencia entre hora programada y hora entregada) y *delay_minutes* (el atraso, nunca negativo).
- Se define *is_on_time* como verdadero cuando el atraso es de 30 minutos o menos.
- Se filtran pesos de paquete inválidos (menores o iguales a 0, o mayores a 10.000 kg).
- Se calculan métricas derivadas: entregas por hora del viaje, eficiencia de combustible (km por litro), costo por entrega y revenue por entrega.
- Se corrige *has_signature* para que sea un valor booleano real, en lugar del texto de la firma.
- Se arman los campos de tipo SCD (Slowly Changing Dimension) — *valid_from*, *valid_to*, *is_current* — para poder llevar historial de cambios en vehículos y conductores.

4.3 Load — Carga en Snowflake

Primero se cargan las seis dimensiones y se obtienen sus *surrogate keys* reales (los identificadores internos que usa el modelo estrella). Con esos mapeos resueltos, se arma e inserta el lote de hechos en *fact_deliveries*, y se descarta —dejando registro en el log— cualquier fila a la que le falte una clave de dimensión válida. Al final del proceso se recalculan los totales diarios (*daily_totals*) para acelerar los reportes.

Todo el proceso queda registrado con *logging* (archivo etl_pipeline.log) y protegido con manejo de errores y *rollback* transaccional en Snowflake, de forma que un error parcial no deje datos inconsistentes cargados.

4.4 Resultado de la última corrida validada

1.800	993	02:00 AM
Registros extraídos	Registros transformados y cargados	Horario de ejecución automática diaria

DATOS EN SNOWFLAKE

5. Ejemplos de las tablas cargadas

A continuación se muestran ejemplos reales de filas de cada tabla del modelo, tal como quedaron cargadas en Snowflake luego de correr el pipeline.

5.1 dim_date

date_key	full_date	day_name	week_of_year	month_name	quarter
20260525	2026-05-25	Monday	22	May	2

Una fila por cada día del calendario, con sus atributos de tiempo ya calculados.

5.2 dim_time

time_key	hour_24	hour_12	time_of_day	is_business_hour	shift
900	09:00	09:00 AM	Mañana	TRUE	Turno 1
1300	13:00	01:00 PM	Tarde	TRUE	Turno 1
2200	22:00	10:00 PM	Noche	FALSE	Turno 3

24 franjas horarias (una por hora) usadas tanto para la hora programada como para la hora real de entrega.

5.3 dim_vehicle

vehicle_key	license_plate	vehicle_type	capacity_kg	fuel_type	age_months	status
2	HHE242	Camión Grande	5000.00	diesel	50	active
96	KYC614	Camión Mediano	3000.00	diesel	60	active
183	AMK812	Van	1500.00	gasolina	19	active

100 vehículos activos entre camiones grandes, medianos y vans, con antigüedad calculada en meses.

5.4 dim_driver

driver_key	employee_code	full_name	hire_date	experience_months	status
341	EMP0341	Camilo Delgado	2024-04-03	27	active
154	EMP0154	Graciela Vergara	2022-10-16	45	active
37	EMP0037	Humberto Cerón	2026-04-30	3	active

395 conductores registrados, con antigüedad en meses calculada al momento de la carga.

5.5 dim_route

route_key	route_code	origin_city	destination_city	distance_km	difficulty_level
19	R019	Medellín	Barranquilla	540.52	Difícil
39	R039	Barranquilla	Cartagena	94.24	Medio
45	R045	Cartagena	Cali	1120.84	Difícil

48 rutas entre las principales ciudades de Colombia, clasificadas por dificultad y tipo (urbana/interurbana/rural).

5.6 dim_customer

customer_key	customer_name	customer_type	city	customer_category
1	Eugenia Alexandra Suárez	Individual	Barranquilla	Regular
3	Juan Manuel Gonzáles	Individual	Bogotá	Regular
17	Catherine Roa	Individual	Cali	Regular

100 clientes distribuidos en Bogotá, Barranquilla, Cartagena y Cali.

5.7 fact_deliveries (tabla de hechos)

delivery_key	date_key	vehicle_key	driver_key	route_key	customer_key	delivery_id
1001	20260525	2	341	19	1	398917
1005	20260525	96	32	8	5	399112
1013	20260525	149	154	42	13	399123

Cada fila conecta una entrega puntual con sus seis dimensiones y guarda además las métricas operativas (peso, distancia, combustible, tiempos, costos e ingresos) no mostradas aquí por espacio.

5.8 Vista v_sales_deliveries (rol Ventas)

full_date	customer_name	customer_type	package_weight_kg	delivery_status	revenue_per_delivery
2026-05-25	Eugenia Alexandra Suárez	Individual	659.33	delivered	349665.00
2026-05-25	Juan Manuel Gonzáles	Individual	889.62	delivered	464810.00
2026-05-25	Simón Barrera Medina	Individual	504.10	delivered	272050.00

Vista pensada para el equipo comercial: solo información de cliente, peso, estado e ingreso — sin datos operativos internos.

5.9 Vista v_operations_deliveries (rol Operaciones)

full_date	hora	license_plate	conductor	route_code	delivery_time_minutes	is_on_time
2026-05-25	02:00	HHE242	Camilo Delgado	R019	145	FALSE
2026-05-25	03:00	KYC614	Alex Acosta	R008	9	TRUE
2026-05-25	06:00	SPQ195	Alejandra Montoya	R009	-14	TRUE

Vista completa para el equipo de Operaciones: incluye vehículo, conductor, ruta, tiempos de entrega y puntualidad.

Este documento resume el trabajo técnico realizado sobre FleetLogix: el diseño del modelo dimensional, la lógica del pipeline ETL en Python y ejemplos reales de los datos ya cargados en Snowflake, como base para el entregable de Arquitectura Técnica del proyecto integrador.